Análisis de movilidad y costes en NYC Taxis

Este proyecto analiza los registros de viajes de los clásicos "Yellow Cabs" de Nueva York, utilizando datos proporcionados directamente por la Taxi & Limousine Commission. La cantidad de información urbana que genera esta flota de vehículos me ha permitido extraer patrones de movilidad y comportamiento económico. El núcleo de este análisis se basa en la separación de las métricas por tipo tiempo y tipo costes, estableciendo un marco de trabajo que permite comprender no solo cómo y cuando se desplazan los ciudadanos, sino también cómo la geografía y el tráfico impactan directamente en la estructura del gasto total de cada viaje.

El stack tecnológico que he utilizado se fundamenta en Python 3, utilizando Pandas y motores como PyArrow para la lectura y cruce de archivos .parquet y .csv. La visualización de datos ha sido implementada con Seaborn para patrones temporales, Plotly para gráficos financieros interactivos, y Geopandas para la representación espacial sobre archivos Shapefile (.shp).

1. ¿Como varía el tiempo medio de duración de un trayecto en función del día de la semana o la hora del día?

El análisis temporal de este gráfico fluctuaciones significativas en la duración de los trayectos dependiendo del día de la semana y la franja horaria. El mapa de calor ilustra cómo los tiempos de viaje aumentan durante las horas punta de los días laborables, coincidiendo con los desplazamientos simétricos (hacia los centros de trabajo por la mañana y vuelta por la tarde). Por el contrario, los fines de semana o las madrugadas muestran duraciones notablemente inferiores como podríamos esperar.

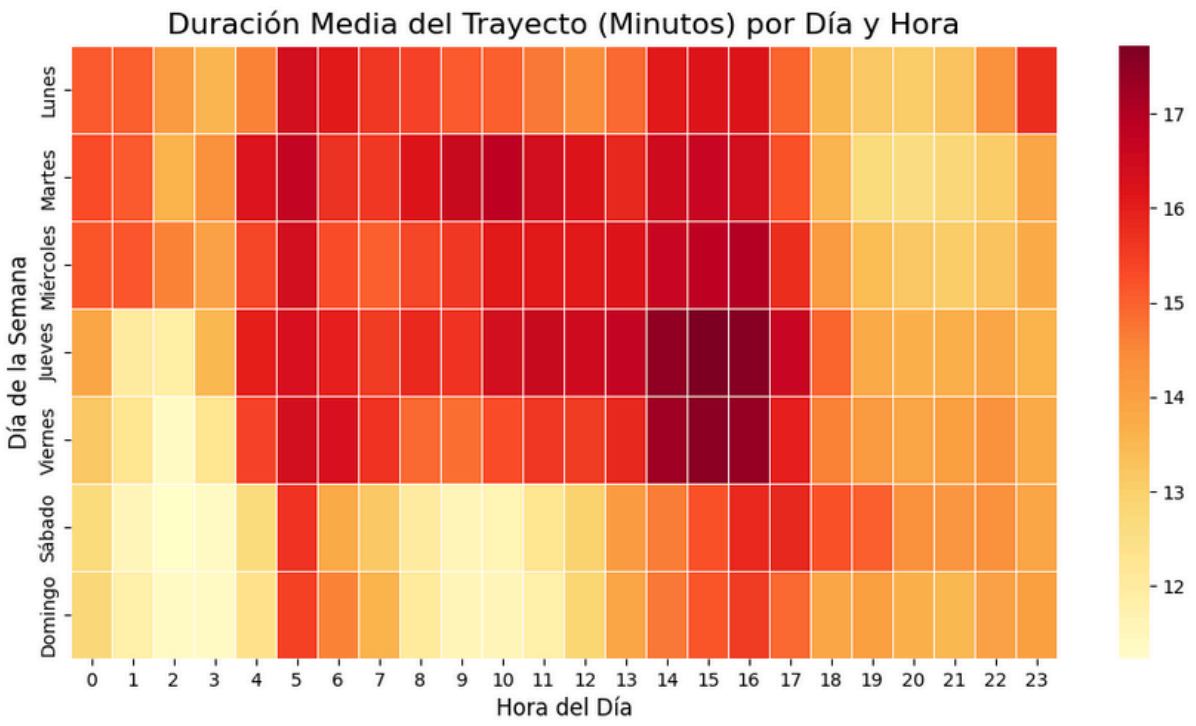

Figura 1: Mapa de calor de viajes

2. ¿Cuales son las tarifas bases, impuestos y propinas por zona de recogida?

Para poder hablar sobre métricas económicas he tenido aislar el coste real del viaje de las propinas y los impuestos. Para evitar sesgos de tamaño de muestra donde barrios residenciales remotos como Charleston mostraban precios altísimos debido a registrar únicamente un puñado mínimo de viajes, he creado un filtro estricto, solo cuentan los barrios con un mínimo de 100 viajes registrados.

El gráfico interactivo de barras apiladas revela que las tarifas más altas se originan en los aeropuertos como JFK, Newark y LaGuardia y en las zonas más alejadas del centro. En estos trayectos, la "Tarifa Base" constituye la inmensa mayoría del importe final.

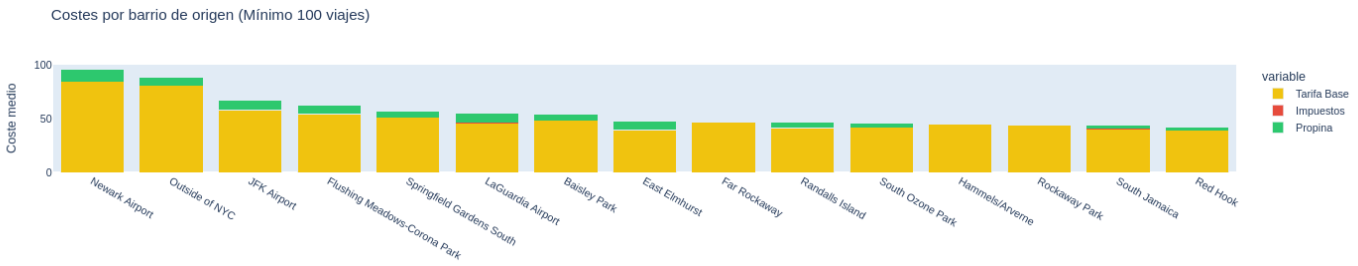

Figura 2: Desglose de costes por barrio de origen

3. ¿Cuales son las zonas donde peor relación hay distancia recorrida/duración, es decir, mayor tráfico?

Para evaluar el impacto espacial del tiempo, he calculado la velocidad media en millas por hora para cada viaje relacionando la duración y la distancia de cada viaje. Durante este cálculo, he detectado y filtrado registros anómalos para preservar la integridad del gráfico. El mapa coroplético resultante demuestra que Manhattan y las zonas interiores de Brooklyn están repletas de tráfico, con velocidades medias que caen hasta las 10-15 mph. En contraste, los distritos periféricos y las rutas hacia los aeropuertos presentan un menor índice de congestión, reflejando velocidades medias superiores a las 35 mph gracias al uso probablemente de autopistas y vías rápidas.

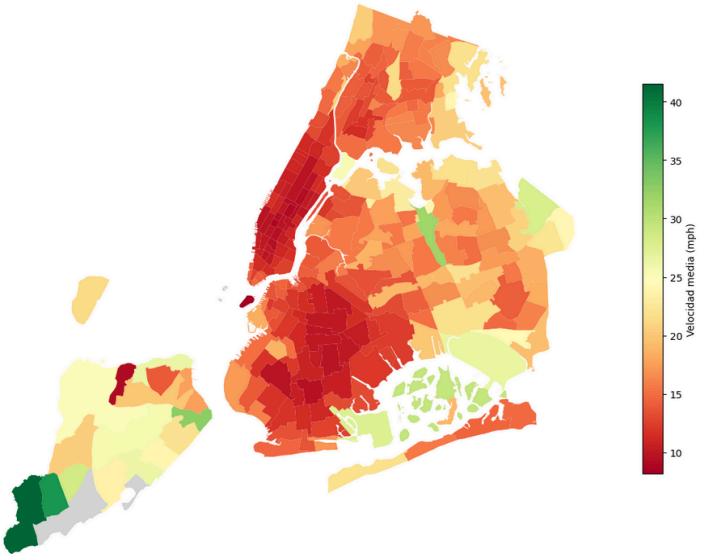

Figura 3: Fluidez y congestión espacial

4. ¿Cómo afecta directamente cada minuto de tráfico al bolsillo del pasajero?

En este gráfico de puntos vemos que, cuanto más dura el viaje, más pagas. Pero lo interesante es ver que hay viajes que duran 20 minutos y cuestan 15 dólares, mientras otros que duran lo mismo suben a 30 dólares. Esto confirma lo que hemos visto en el mapa, no es lo mismo estar 20 minutos atrapado en un semáforo de Manhattan que ir conduciendo por la autopista hacia el aeropuerto.

Como observación de interés, vemos que varios viajes que abarcan casi todo el espectro temporal cuestan lo mismo, se puede apreciar concretamente dentro de la figura en el segmento horizontal que se forma en los 70 dólares, esto puede deberse a tarifas planas de trayectos desde aeropuertos a hoteles o a puntos relacionados con el sector de hospitality.

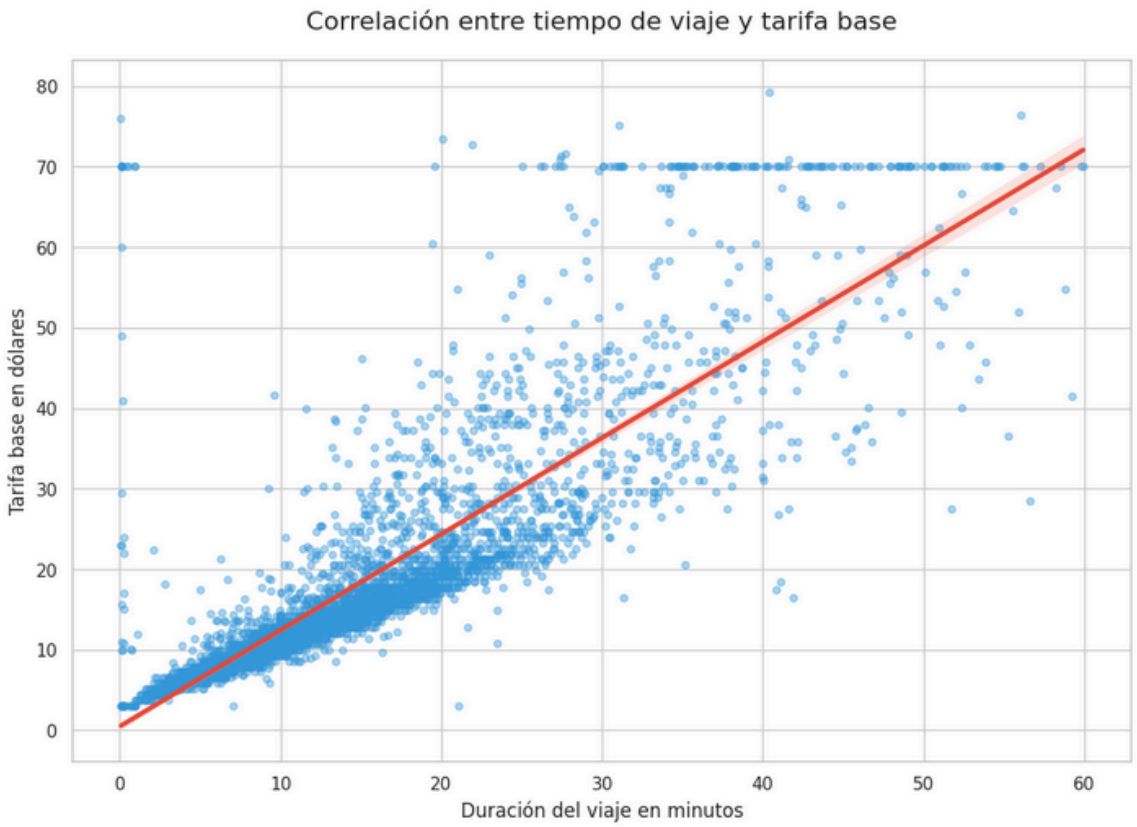

Figura 4: Correlación tiempo vs duración

Bibliografía

- Página oficial del gobierno de NY: https://www.nyc.gov/site/tlc/
- Guía de features del dataset: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
- Página oficial de Plotly: https://plotly.com/python/
- Como hacer un mapa de calor en Seaborn: https://www.datacamp.com/es/tutorial/seaborn-heatmaps

Uso de IA

En primer lugar he utilizado Gemini para aprender sobre fusión de sets de datos con pandas ya que es algo que no sabía como hacer bien y se trataba de un aspecto clave de la práctica:

Unir Datasets Parquet con Pandas

Cambiar a

tengo un dataset en formato parquet en mi /content de google collab, quiero hacer un fetch a una api externa para descargar otro dataset que esta en formato parquet, quiero convertirlo a .csv y luego concatenarlo al dataset que tengo ya descargado, quiero hacerlo co...

Ver razonamiento

¡Hola! Es un flujo de trabajo excelente para Google Colab.

Antes de entrar en las diferencias, te doy **un pequeño consejo práctico**: si tu objetivo final es unir ambos datasets en Pandas, **no necesitas convertir el segundo dataset a** `.csv`. Pandas lee `.parquet` de forma nativa y muchísimo más rápido que los CSV. Además, el formato Parquet preserva los tipos de datos (como fechas o enteros), algo que se suele romper o desconfigurar al pasarlo a CSV. Lo ideal es cargar ambos Parquet directamente a DataFrames y unirlos.

Dicho esto, la elección entre `.concat`, `.merge` y `.join` depende completamente de **cómo** quieres que se unan esos datos. Aquí tienes la diferencia clara entre los tres:

1. `pd.concat()`: "Pegar con cinta adhesiva"

Utiliza `concat` cuando quieras apilar o "pegar" DataFrames de forma estructural, ya sea apilando filas (uno encima del otro) o pegando columnas (uno al lado del otro), sin buscar que los valores coincidan lógicamente.

- **Cuándo usarlo:** Cuando descargas datos nuevos que tienen las **mismas columnas** que tu dataset original y simplemente quieres añadir esos registros nuevos al final (añadir más filas).
- **Ejemplo visual:** Tienes los datos de ventas de enero y descargas los de febrero. Los

Python

```python
import pandas as pd
import requests
import io

# 1. Cargar tu dataset actual desde Colab
df_local = pd.read_parquet('/content/tu_dataset_local.parquet')

# 2. Hacer fetch a la API externa
url_api = "URL_DE_TU_API_AQUI"
response = requests.get(url_api)

# 3. Leer el parquet directamente desde la memoria (los bytes de la respuesta)
# (Si la API devuelve una URL de descarga, descargarías el archivo y pondrías la
df_externo = pd.read_parquet(io.BytesIO(response.content))

# 4. Concatenar (apilar uno debajo del otro)
# ignore_index=True resetea la numeración de las filas para que sea continua
df_final = pd.concat([df_local, df_externo], ignore_index=True)

# Ver el resultado
print(df_final.head())
```

¿Los datos que vas a descargar de la API tienen las mismas columnas que tu dataset actual (para apilarlos), o traen información nueva sobre los mismos registros que necesitas cruzar?

traen las mismas columnas, solo que en otro segmento de tiempo, hay un id único autoincremental que independiza cada registro individual, pero en lo que me has dado no veo o no entiendo donde se hace la conversión al formato .csv

Como me confirmas que es el mismo formato pero en otro segmento de tiempo, `pd.concat()` **es sin duda la herramienta correcta.** Aquí tienes el código actualizado, esta vez incluyendo el paso final para exportar tu gran tabla unida a un archivo `.csv`:

Python

```python
import pandas as pd
import requests
import io

# 1. Cargas tu dataset actual
df_local = pd.read_parquet('/content/tu_dataset_local.parquet')

# 2. Descargas el nuevo segmento de tiempo de la API
url_api = "URL_DE_TU_API_AQUI"
response = requests.get(url_api)

# 3. Lo lees en memoria manteniendo el formato rápido y seguro (Parquet)
df_externo = pd.read_parquet(io.BytesIO(response.content))

# 4. Los apilas uno debajo del otro
df_final = pd.concat([df_local, df_externo], ignore_index=True)

# 5. ¡AQUÍ ESTÁ LA CONVERSIÓN A CSV!
# Guardas el resultado final combinado como un archivo .csv en tu Colab
# index=False evita que se guarde una columna extra con la numeración de las fila
df_final.to_csv('/content/dataset_combinado_final.csv', index=False)

print("¡Dataset combinado y guardado como CSV con éxito!")
```

Tras estos mensajes le pregunté como podría añadir las localizaciones en formato texto haciendo un left join entre mi dataset tal y como lo tenía y agregando solo las filas de texto de localización cuyos LocationID coincidieran con los PUId's de los taxis, para no tener zonas almacenadas que no se fueran a usar.

La IA alucinó un poco y entre búsqueda y búsqueda escribí este fragmento:

```
df_con_nombres = pd.merge(
    df_taxis,
    df_diccionario,
    left_on='PULocationID',
    right_on='LocationID',
    how='left'
)
```

A continuación le pregunto acerca del mapa coroplético, el cual no sabía como empezar y la documentación de geopandas no me ayudó ya que es muy extensa y variada.

Unir Datasets Parquet con Pandas

Un apunte sobre tu ID autoincremental:
Como mencionas que tienes un ID único que independiza cada registro, al hacer el `concat` simplemente se apilarán. Si los nuevos datos de la API ya traen sus propios IDs correctos (por ejemplo, el local termina en el ID 1000 y la API te trae del 1001 en adelante), el `concat` los dejará perfectos. El parámetro `ignore_index=True` del código solo resetea el índice interno de Pandas, sin tocar para nada tu columna de ID autoincremental.

necesito ayuda para crear un mapa coroplético, de momento solo he limpiado los datos y les he dado el formato que me interesa: df_mapa = df.copy()
df_mapa = df_mapa[(df_mapa['duracion_minutos'] > 0) & (df_mapa['trip_distance'] > 0)]
df_mapa['velocidad_mph'] = df_mapa['trip_distance'] / (df_mapa['duracion_minutos'] / 60)df_mapa = df_mapa[df_mapa['velocidad_mph'] <= 80]
velocidad_zona = df_mapa.groupby('PULocationID')['velocidad_mph'].mean().reset_index() tengo descargados unos archivos .shp .shx .prj .dbf y .cpg, quiero saber a que corresponden cada uno y que me ayudes con la creación de este mapa explicándome cada detalle

1. La anatomía de tu Shapefile

- `.shp` **(Shape):** Es el archivo principal y el que contiene la geometría. Aquí están guardadas las coordenadas exactas que dibujan las fronteras de cada zona o barrio.
- `.dbf` **(Database):** Es una tabla de datos (como un Excel antiguo). Contiene los "atributos" de cada zona. Por ejemplo, el nombre del barrio, a qué distrito pertenece y, lo más importante, el ID de la zona para poder cruzarlo con tus datos.
- `.shx` **(Shape Index):** Es un índice que conecta la geometría (`.shp`) con los datos de la tabla (`.dbf`). Permite que Python encuentre rápidamente qué polígono corresponde a qué fila de datos.
- `.prj` **(Projection):** Contiene el "Sistema de Referencia de Coordenadas" (CRS). Básicamente, le dice al ordenador si esas coordenadas están en latitud/longitud tradicionales o en un mapa plano medido en metros o pies.
- `.cpg` **(Codepage):** Es un archivo pequeñito que le dice al sistema cómo leer los caracteres de texto (para que los acentos o las "ñ" en el `.dbf` no se vean como símbolos raros).

Paso 2: Cargar el mapa

El truco aquí es que, aunque tengas 5 archivos, **solo le pasas a Geopandas la ruta del archivo** `.shp`. Geopandas es lo suficientemente inteligente como para leer automáticamente el `.dbf`, `.prj`, etc., siempre y cuando estén en la misma carpeta.

Python

```python
# Sube todos los archivos (.shp, .shx, .dbf, .prj, .cpg) a una carpeta en tu Cola
# Leemos solo el .shp
mapa_zonas = gpd.read_file('/content/ruta_a_tu_carpeta/tu_archivo.shp')

# Echa un vistazo a lo que acabas de cargar.
# Verás que es un DataFrame normal, pero con una columna especial llamada 'geomet
print(mapa_zonas.head())
```

Paso 3: Unir tus datos con el mapa

Ahora tenemos que cruzar tu DataFrame `velocidad_zona` con `mapa_zonas`. Para hacer esto, necesitas identificar cómo se llama la columna de ID en el shapefile (normalmente en los datos de taxis de NYC se llama `LocationID`).

Python

```python
# Hacemos un merge (como el que hablamos antes)
# Suponiendo que en mapa_zonas la columna se llama 'LocationID'
mapa_completo = mapa_zonas.merge(velocidad_zona,
                                 left_on='LocationID',
                                 right_on='PULocationID',
                                 how='left')
```

Estos pasos son los pasos que más tiempo me han podido ahorrar para este gráfico, aunque la lógica pueda parecer simple (leer y fusionar) en mi caso no sabía para que servía cada fichero, cual o cuales había que leer, como se cruzan las localizaciones para que cada una tenga sus coordenadas dentro del mapa ect.

En general estos han sido los puntos clave donde he realmente necesitado usar la IA, para puntualidades pequeñas o pequeños errores cometidos por mí suelo acudir a duck.ai que es un LLM rápido y no muy preciso pero me sirve igual para pequeñas consultas. Por lo demás me he apoyado en tutoriales, documentaciones oficiales, blogs como datacamp y en conocimiento propio adquirido durante la carrera y aprendizaje propio.